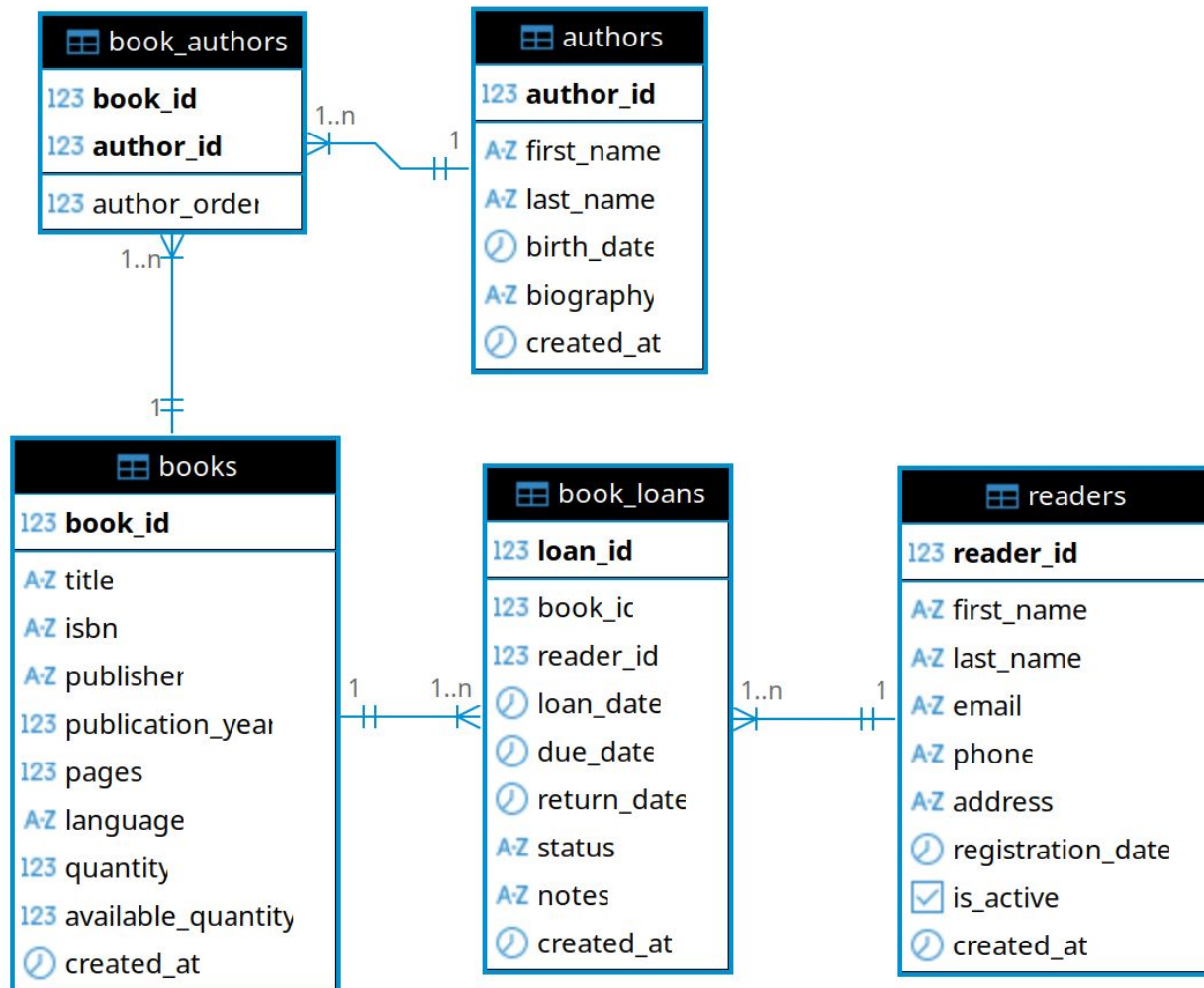


Выборка данных. Соединения.

Лекция 5



Расширенные возможности SELECT

Ограничения выборки

`SELECT title, pages FROM books ORDER BY pages DESC LIMIT 3;` – топ 3 самых толстых книги

Пагинация:

Первая страница (первые 5 записей):

`SELECT title, publication_year FROM books ORDER BY title LIMIT 5 OFFSET 0;`

Вторая страница (пропустить 5, показать следующие 5):

`SELECT title, publication_year FROM books ORDER BY title LIMIT 5 OFFSET 5;`

Третья страница (пропустить 10, показать следующие 5):

`SELECT title, publication_year FROM books ORDER BY title LIMIT 5 OFFSET 10`

Выражения IN, ANY/SOME, ALL

выражение **IN** (*значение* [, ...]) эквивалентно

выражение = *значение1* OR *выражение* = *значение2* OR ...

выражение **NOT IN** (*значение* [, ...]) эквивалентно

выражение <> *значение1* AND *выражение* <> *значение2* AND ...

выражение оператор ANY (*выражение массива*) – сравнивает с помощью оператора результат левого выражения с каждым элементом массива: вернет true, если хотя бы одно сравнение true, и false в противном случае. Если все сравнения false, но есть хотя бы один NULL, то вернет NULL.

выражение оператор ALL (*выражение массива*) – сравнивает с помощью оператора результат левого выражения с каждым элементом массива: вернет true, если для всех элементов сравнение true, и false в противном случае. Если все сравнения true, но есть хотя бы один NULL, то вернет NULL.

Выражения IN, ANY, ALL

IN эквивалентно = ANY

```
SELECT * FROM books WHERE publisher IN ('АСТ', 'Азбука', 'Махаон');  
SELECT * FROM books WHERE publisher = ANY (ARRAY['АСТ', 'Азбука', 'Махаон']);
```

NOT IN эквивалентно != ALL

```
SELECT * FROM books WHERE publisher NOT IN ('АСТ', 'Махаон');  
SELECT * FROM books WHERE publisher != ALL (ARRAY['АСТ', 'Махаон']);
```

> ANY эквивалентно “больше минимального”

```
SELECT * FROM books WHERE pages > ANY (ARRAY[128, 256, 320]); -- > 128
```

> ALL эквивалентно “больше максимального”

```
SELECT * FROM books WHERE pages > ALL (ARRAY[128, 256, 320]); -- > 320
```

Ключевое слово DISTINCT

`SELECT DISTINCT status FROM book_loans;` – уникальные статусы выдачи

`SELECT DISTINCT publisher FROM books WHERE publisher IS NOT NULL ORDER BY 1 LIMIT 3;` – список 3-х уникальных издательств, отсортированных по алфавиту

`SELECT DISTINCT publication_year FROM books WHERE publication_year IS NOT NULL ORDER BY 1 DESC;` – список годов публикации

`SELECT DISTINCT publisher, publication_year FROM books ORDER BY publisher, publication_year DESC;` – список уникальных пар издатель-год публикации

Оператор LIKE

```
SELECT title, isbn, publication_year FROM books WHERE title LIKE 'Гарри%';
```

– поиск книг, у которых название начинается со слова “Гарри”

```
SELECT title, publisher, publication_year FROM books WHERE title LIKE '%душ_';
```

– поиск книг, у которых название заканчивается на “душ” с одним любым символом справа (например, “души”)

```
SELECT title, publisher FROM books WHERE title ILIKE '%война%'; –
```

регистронезависимый поиск книг, у которых в названии есть “война”

% – ноль, один или несколько символов

_ – ровно один символ

Внимание! % в начале шаблона отключает индекс по столбцу

Также существует оператор NOT LIKE

Регулярные выражения

`SELECT title, isbn FROM books WHERE title ~ '[0-9]';` — ищем цифры

`SELECT first_name, last_name FROM authors WHERE last_name ~ '^[A-Г]';` — ищем авторов, у которых фамилия начинается с букв А, Б, В или Г

- `~` соответствует (регистрозависимый)
- `~*` соответствует (регистронезависимый)
- `!~` не соответствует (регистрозависимый)
- `!~*` не соответствует (регистронезависимый)

Также существует более простой вариант регэкспов (поддерживаются не все возможности POSIX) с оператором SIMILAR TO

<https://postgrespro.ru/docs/postgresql/current/functions-matching>

Вычисляемые столбцы и псевдонимы

```
SELECT title, quantity, available_quantity,  
quantity - available_quantity AS books_on_loan, available_quantity * 100.0 /  
quantity AS availability_percent FROM books WHERE quantity > 0; – СЧИТАЕМ  
СКОЛЬКО КНИГ ВЫДАНО И СКОЛЬКО ДОСТУПНО
```

```
SELECT first_name || ' ' || last_name AS full_name FROM readers ORDER BY  
full_name; – конкатенация строк. Также есть функция CONCAT
```

```
SELECT title, pages, ROUND(pages / 10.0) * 10 pages_rounded_to_10, CEIL(pages  
/ 100.0) estimated_hours_to_read FROM books WHERE pages IS NOT NULL; – МОЖНО  
задавать псевдонимы и без AS
```

```
SELECT book_id, loan_date, COALESCE(return_date::TEXT, 'Не возвращена') AS  
return_status, COALESCE(notes, 'Без комментариев') AS comments FROM book_loans  
ORDER BY loan_date DESC; – замена NULL
```

Конструкция CASE - THEN

Простой IF-ELSE:

```
SELECT title, quantity, available_quantity,  
CASE  
    WHEN available_quantity > 0 THEN 'Доступна'  
    ELSE 'Все экземпляры выданы' ←  
END AS availability_status FROM books;
```

ELSE необязателен, но лучше с ним. Если ни одно условие не сработает – вернется NULL

Категоризация:

```
SELECT title, pages,  
CASE  
    WHEN pages < 200 THEN 'Тонкая (< 200 стр.)'  
    WHEN pages BETWEEN 200 AND 400 THEN 'Средняя (200-400 стр.)'  
    WHEN pages BETWEEN 401 AND 700 THEN 'Толстая (400-700 стр.)'  
    WHEN pages > 700 THEN 'Очень толстая (> 700 стр.)'  
    ELSE 'Неизвестно'  
END AS book_size FROM books ORDER BY pages;
```

Шпаргалка основных функций

CONCAT(str1, str2, ...) или
str1 || str2 – Объединение строк
UPPER(string) – В верхний регистр
LOWER(string) – В нижний регистр
LENGTH(string) – Длина строки
TRIM(string) – Убрать пробелы с краев
SUBSTRING(string FROM n FOR len) – Подстрока
REPLACE(string, from, to) – Замена подстроки
LEFT(string, n) – Первые n символов
RIGHT(string, n) – Последние n символов

CURRENT_DATE – Текущая дата
CURRENT_TIMESTAMP – Текущие дата и время
NOW() – То же что CURRENT_TIMESTAMP
AGE(date1, date2) – Разница между датами
EXTRACT(YEAR FROM date) – Извлечь
год/месяц/день
DATE_PART('year', date) – То же что EXTRACT
DATE_TRUNC('month', timestamp) – Усечь до
месяца/дня/часа

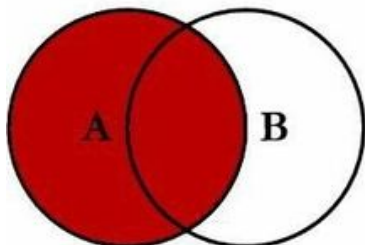
ROUND(number, decimals) – Округление
CEIL(number) – Округление вверх
FLOOR(number) – Округление вниз
ABS(number) – Абсолютное значение
RANDOM() – Случайное число 0-1
GREATEST(val1, val2, ...) – Максимальное значение
LEAST(val1, val2, ...) – Минимальное значение

CAST(value AS type) – Преобразование типа
value::type – Краткая форма
TO_CHAR(value, format) – В строку с форматом
TO_DATE(string, format) – В дату
TO_NUMBER(string, format) – В число

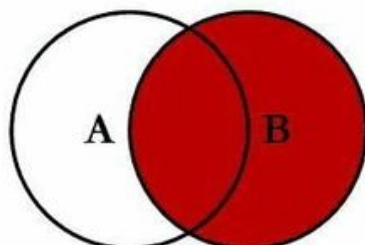
COALESCE(val1, val2, ...) – Первое не-NULL значение
NULLIF(val1, val2) – NULL если равны

Соединения (joins)

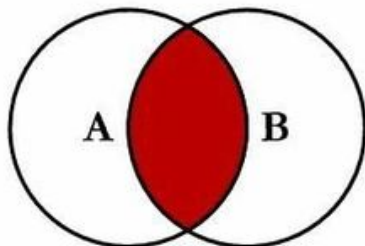
SQL JOINS



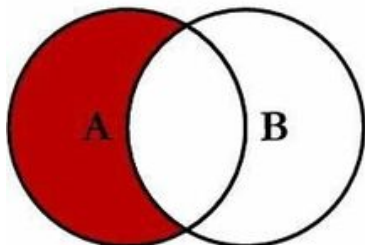
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



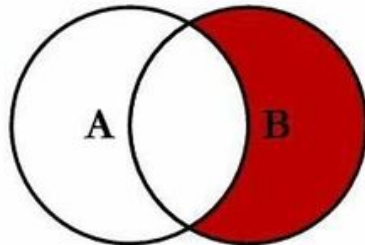
```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



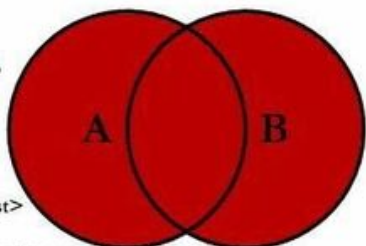
```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



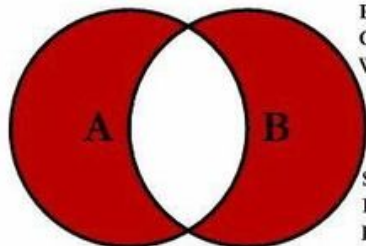
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```



```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```

Многотабличные запросы

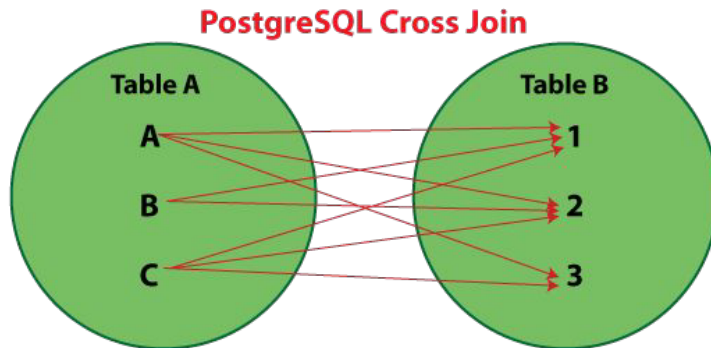
CROSS JOIN: Перекрестное соединения, возвращает комбинации каждой записи первой таблицы с каждой записью второй таблицы.

INNER JOIN: Внутреннее соединение – это перекрестное соединение, из результатов которого часть записей исключена условием.

LEFT (OUTER) JOIN: В левом внешнем соединении для каждой записи левой таблицы ищется соответствие правой по условию. Если запись в правой таблице отсутствует, то значения заменяются на NULL.

RIGHT (OUTER) JOIN: Как левое, но наоборот:)

CROSS JOIN



```
SELECT r.first_name || ' ' || r.last_name AS reader_name, b.title AS  
book_title FROM readers r CROSS JOIN books b; – ВСЕ ВОЗМОЖНЫЕ КОМБИНАЦИИ  
ЧИТАТЕЛЕЙ И КНИГ
```

```
SELECT r.first_name || ' ' || r.last_name AS reader_name, b.title AS  
book_title FROM readers r, books b; – альтернативный синтаксис без CROSS JOIN
```

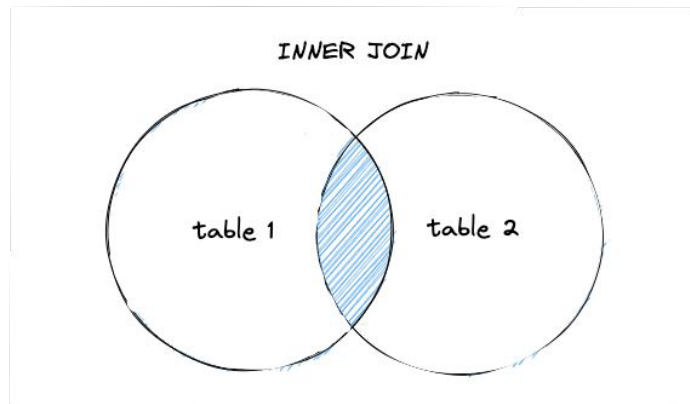

CROSS JOIN

Создать расписание – каждая книга для каждого дня недели:

```
SELECT b.title, days.day_name, days.day_number
FROM books b
CROSS JOIN (
    VALUES
        (1, 'Понедельник'),
        (2, 'Вторник'),
        (3, 'Среда'),
        (4, 'Четверг'),
        (5, 'Пятница')
) AS days(day_number, day_name)
WHERE b.book_id <= 3  -- возьмем только 3 книги
                        для примера
ORDER BY b.book_id, days.day_number;
```

	title	day_name	day_number
1	Война и мир	Понедельник	1
2	Война и мир	Вторник	2
3	Война и мир	Среда	3
4	Война и мир	Четверг	4
5	Война и мир	Пятница	5
6	Анна Каренина	Понедельник	1
7	Анна Каренина	Вторник	2
8	Анна Каренина	Среда	3
9	Анна Каренина	Четверг	4
10	Анна Каренина	Пятница	5
11	Преступление и наказание	Понедельник	1
12	Преступление и наказание	Вторник	2
13	Преступление и наказание	Среда	3
14	Преступление и наказание	Четверг	4

INNER JOIN



Все читатели и их выдачи:

```
SELECT b1.loan_id, b1.loan_date, r.first_name, r.last_name  
FROM book_loans b1 INNER JOIN readers r ON b1.reader_id = r.reader_id;
```

Или без INNER JOIN:

```
SELECT b1.loan_id, b1.loan_date, r.first_name, r.last_name  
FROM book_loans b1, readers r WHERE b1.reader_id = r.reader_id
```

INNER JOIN

Читатели и книги, которые сейчас читают:

```
SELECT r.first_name, r.last_name, b.title, bl.loan_date, bl.due_date
FROM readers r
INNER JOIN book_loans bl ON r.reader_id = bl.reader_id
JOIN books b ON bl.book_id = b.book_id
WHERE bl.status = 'active';
```

Можно опустить **INNER**

	first_name	last_name	title	loan_date	due_date
1	Иван	Иванов	1984	2024-12-14	2024-12-28
2	Мария	Петрова	451 градус по Фаренгейту	2024-12-15	2024-12-29
3	Алексей	Смирнов	Гарри Поттер и философский камень	2024-12-16	2024-12-30
4	Елена	Кузнецова	Гарри Поттер и Тайная комната	2024-12-17	2024-12-31
5	Виктор	Михайлов	Война и мир	2024-12-01	2024-12-15
6	Наталья	Васильева	Анна Каренина	2024-12-05	2024-12-19
7	Андрей	Егоров	Преступление и наказание	2024-12-08	2024-12-22
8	Светлана	Павлова	Братья Карамазовы	2024-12-10	2024-12-24
9	Максим	Семёнов	Мастер и Маргарита	2024-12-12	2024-12-26

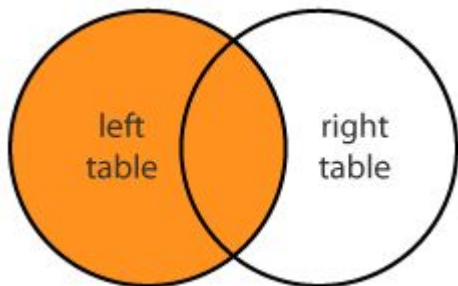
JOIN с другими операторами (!=, <, >, LIKE и др.)

Найдем пары книг, где одна больше другой:

```
SELECT
    b1.title AS thin_book,
    b1.pages AS thin_pages,
    b2.title AS thick_book,
    b2.pages AS thick_pages,
    (b2.pages - b1.pages) AS difference
FROM books b1
JOIN books b2 ON b1.pages < b2.pages
WHERE b1.publisher = b2.publisher
ORDER BY difference DESC
LIMIT 3;
```

	□ thin_book ∇	÷	□ thin_pages ∇	÷	□ thick_book ∇	÷	□ thick_pages ∇	÷	□ difference ∇	÷
1	Скотный двор		128		Война и мир		1300		1172	
2	Старик и море		128		Война и мир		1300		1172	
3	1984		320		Война и мир		1300		980	

LEFT JOIN



Все читатели и даты возврата книг:

SELECT

```
  r.first_name,  
  r.last_name,  
  bl.return_date
```

FROM readers r

LEFT JOIN book_loans bl

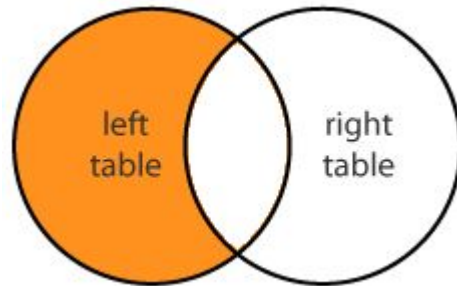
ON r.reader_id = bl.reader_id;

	first_name ▼	last_name ▼	return_date ▼
1	Иван	Иванов	2024-01-22
2	Мария	Петрова	2024-01-28
3	Алексей	Смирнов	2024-02-14
4	Елена	Кузнецова	2024-02-25
5	Дмитрий	Попов	2024-02-23
6	Анна	Соколова	2024-03-05
7	Сергей	Морозов	2024-03-04
8	Ольга	Волкова	2024-03-14
9	Павел	Новиков	2024-03-18
10	Татьяна	Федорова	2024-03-23
11	Виктор	Михайлов	<null>
12	Наталья	Васильева	<null>
13	Андрей	Егоров	<null>
14	Светлана	Павлова	<null>

LEFT JOIN С ФИЛЬТРОМ IS (NOT) NULL

Книги, которые никогда не брали:

```
SELECT b.title, b.isbn, b.quantity
FROM books b
LEFT JOIN book_loans bl ON b.book_id = bl.book_id
WHERE bl.loan_id IS NULL;
```



Читатели, которые ХОТЯ БЫ РАЗ брали книги:

```
SELECT
    DISTINCT (r.first_name || ' ' || r.last_name) AS reader_name
FROM readers r
LEFT JOIN book_loans bl ON r.reader_id = bl.reader_id
WHERE bl.loan_id IS NOT NULL
ORDER BY reader_name; — МОЖНО заменить на INNER JOIN без условия
```

ON vs WHERE

ВСЕ читатели и книги, которые они взяли:

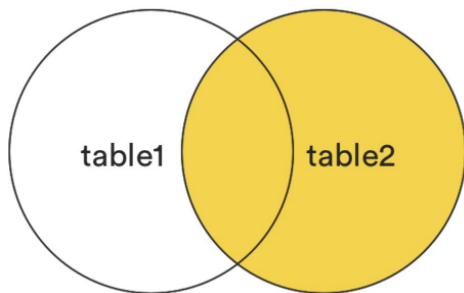
```
SELECT r.first_name, r.last_name, b.title AS borrowed_book
FROM readers r
LEFT JOIN book_loans bl ON r.reader_id = bl.reader_id AND bl.status = 'active'
LEFT JOIN books b ON bl.book_id = b.book_id;
```

Только читатели с активными выдачами:

```
SELECT r.first_name, r.last_name, b.title AS borrowed_book
FROM readers r
LEFT JOIN book_loans bl ON r.reader_id = bl.reader_id
LEFT JOIN books b ON bl.book_id = b.book_id
WHERE bl.status = 'active';
```

– аналогично запросу с INNER JOIN, потому что не может быть нулевой строки book_loan со статусом 'active'

RIGHT JOIN

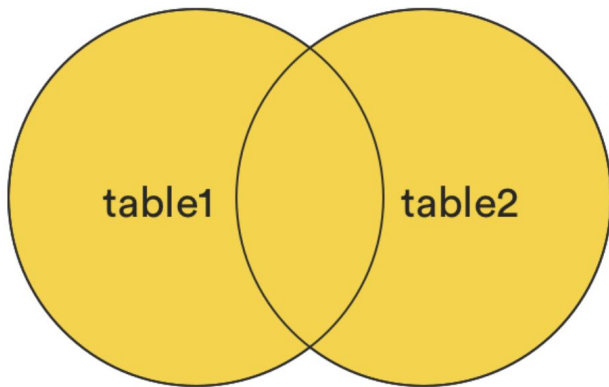


Все выдачи и информация о читателях (даже если читатель удален):

```
SELECT r.first_name, r.last_name, bl.loan_id, bl.loan_date, bl.status
FROM readers r
RIGHT JOIN book_loans bl ON r.reader_id = bl.reader_id;
```

Так пишут редко, лучше переделать в LEFT JOIN!

FULL OUTER JOIN



- FULL OUTER JOIN возвращает ВСЕ строки из ОБЕИХ таблиц
- Где есть совпадение - данные из обеих таблиц
- Где нет совпадения - NULL с одной из сторон

Изи!

Какой запрос вернет больше строк?

```
SELECT r.first_name || ' ' || r.last_name AS reader_name,  
       bl.loan_id, bl.loan_date, bl.status  
FROM readers r  
FULL OUTER JOIN book_loans bl ON r.reader_id = bl.reader_id  
ORDER BY reader_name;
```

VS

```
SELECT r.first_name || ' ' || r.last_name AS reader_name,  
       bl.loan_id, bl.loan_date, bl.status  
FROM readers r  
FULL OUTER JOIN book_loans bl ON r.reader_id = bl.reader_id  
   AND bl.status = 'active'  
ORDER BY reader_name;
```

ОТВЕТ

```
SELECT r.first_name || ' ' || r.last_name AS reader_name,  
       bl.loan_id, bl.loan_date, bl.status  
FROM readers r  
FULL OUTER JOIN book_loans bl ON r.reader_id = bl.reader_id  
ORDER BY reader_name;
```

vs

ВОТ ЭТОТ ВЕРНЕТ БОЛЬШЕ

```
SELECT r.first_name || ' ' || r.last_name AS reader_name,  
       bl.loan_id, bl.loan_date, bl.status  
FROM readers r  
FULL OUTER JOIN book_loans bl ON r.reader_id = bl.reader_id  
    AND bl.status = 'active'  
ORDER BY reader_name;
```



Как это понять?

Представим, что у нас в фильтре стоит `AND b1.status = 'not_existing_status'`. Тогда запрос выдаст количество строк равное сумме строк в обеих таблицах – максимальное количество строк запроса `FULL OUTER JOIN`

Теперь, меняя статус на корректный `AND b1.status = 'active'`, часть строк из обеих таблиц попадают в когорту “ненуловых” строк. Те, кто не попал туда представляют строки с нулями у соответствующей стороны.

Убирая фильтр по статусу, мы еще больше расширяем эту когорту, уменьшая количество строк с нулями.

Операции над выборками

- UNION – объединение множеств, строки из обоих множеств включаются в результат
- INTERSECT – пересечение множеств, строка включается в результат, если присутствует в каждом множестве
- EXCEPT – разность множеств, строка включается в результат, если присутствует в левом множестве, но отсутствует в правом

Для выполнения операций запросы должны возвращать одинаковое число столбцов и типы данных столбцов должны совпадать

UNION

Объединить имена читателей и авторов:

```
SELECT first_name, last_name, 'Читатель' AS type
FROM readers
```

UNION

```
SELECT first_name, last_name, 'Автор' AS type
FROM authors
ORDER BY last_name;
```

UNION – убирает дубликаты

UNION ALL – оставляет дубликаты

	first_name	last_name	type
1	Рэй	Брэдли	Автор
2	Михаил	Булгаков	Автор
3	Наталья	Васильева	Читатель
4	Ольга	Волкова	Читатель
5	Николай	Гоголь	Автор
6	Фёдор	Достоевский	Автор
7	Андрей	Егоров	Читатель
8	Сергей	Есенин	Автор
...	...	...	...

FULL OUTER JOIN через UNION

SELECT

```
r.first_name || ' ' || r.last_name AS reader_name,  
bl.loan_id, bl.loan_date, bl.status
```

FROM readers r

LEFT JOIN book_loans bl ON r.reader_id = bl.reader_id

UNION

SELECT

```
r.first_name || ' ' || r.last_name AS reader_name,  
bl.loan_id, bl.loan_date, bl.status
```

FROM readers r

RIGHT JOIN book_loans bl ON r.reader_id = bl.reader_id

ORDER BY reader_name;

INTERSECT

Найдем новые и толстые книги:

```
SELECT title, publication_year, pages
FROM books
WHERE publication_year >= 2020
```

INTERSECT

```
SELECT title, publication_year, pages
FROM books
WHERE pages >= 400;
```

Конечно можно сделать это через WHERE ... AND ...

EXCEPT

Читатели, которые не брали книги:

```
-- Все читатели
```

```
SELECT reader_id, first_name, last_name  
FROM readers
```

EXCEPT

```
-- Читатели, которые брали книги
```

```
SELECT DISTINCT r.reader_id, r.first_name, r.last_name  
FROM readers r  
JOIN book_loans bl ON r.reader_id = bl.reader_id;
```